Westside Stylist Mirror: baselines first, then models

ANCP D4 · Modelling plan and experiments · Group M · C Prashanth · Linked outcome CO3 · Data: data-v1.0 (D2), H&M 10% customer sample as a proxy for Westside purchase history · Code and log: github.com/cprashanth666666-sys/Captain, folder D4/, tag d4-v1

Bottom line

A learned ranker (LightGBM LambdaRank) gives the largest and only tested-significant gain over the simple rule. On the untouched test week it scores **MAP@12 0.0308** vs **0.0266** for 'show the customer's own recent items, then best-sellers' (B1), a gain of +0.0042 (paired bootstrap 95% CI +0.0022 to +0.0063, 6,820 customers). The tuned co-purchase scorer (M1) does **not** beat B1 on test (difference -0.0004, CI -0.0016 to +0.0007): its small validation edge was tuning noise. Pure best-sellers (B0) score 0.0082, so most of the signal is re-buying, as D3 insight I3 predicted. Absolute scores are low because only 9.6% of next-week purchases ever enter the candidate pool; candidate recall, not the ranker, is the next bottleneck.

1 Problem type and why these model families fit

The mirror must show a short list of items a shopper is likely to buy next. We frame this as **top-12 next-week recommendation from implicit feedback**: a ranking problem, not classification or regression, because only the order of the first few items shown matters and the label is a purchase, never a rating. MAP@12 is the metric of the source competition [1] and rewards putting the right items first.

Family	Why it fits this problem (evidence from D3)	Risk we watch
Popularity / repurchase rules (B0, B1)	Demand turns over within weeks (I5) and exact re-buys are 19× random (I3), so a recency rule is a strong, explainable floor. Literature shows tuned rules often beat complex recommenders [7].	Ignores cross-sell; serves everyone the same list (B0).
Item-to-item co-purchase (M1)	Item-based CF [4] scales to a store catalogue and covers cross-sell; D3 found co-purchase useful as a candidate source (I4).	Co-purchase cannot tell complements from substitutes (I4).
Linear ranker (M2)	Logistic regression on the same 20 numeric features tests whether learning weights beats hand-set weights before adding non-linearity.	Cannot model interactions (e.g. recency × product type).
Gradient-boosted LambdaRank (M3)	LambdaMART optimises a ranking loss directly [5]; LightGBM [6] handles mixed numeric / categorical features, missing values and 0.6 M rows on a laptop.	Over-fitting the validation week; opaque to staff.

2 Validation design, declared before tuning

Fixed in **validation_design.json** on 2026-11-12 before the first model run. Its hash (e3cb0002dafc) is stamped on all 52 log rows. The split is **by date**, because the mirror will always predict a future week from the past; a random or customer-level split would let the model see the future (fashion demand drifts weekly, D3 I5) [2][3]. D3 had suggested a customer split; we keep a time split instead and report results by activity tier, which covers the heavy-buyer concern (I6) without leaking time.

Part	Dates	Used for
Train	2018-09-20 to 2020-09-08	History for features; four weekly target windows (Aug 12 to Sep 8) to train M2 and M3
Validation	2020-09-09 to 2020-09-15	All tuning and selection (47 runs)
Test	2020-09-16 to 2020-09-22	Scored once with frozen settings (5 runs); a lock file blocks a second run

Leakage guards: every feature for a week starting on day S uses rows dated before S only; customer totals from D2 that were computed on the full train window are recomputed week by week; basket_units, flagged as leakage-prone in the D2 dictionary, is excluded. For the test week the rankers are refit on the same four-week scheme shifted one week later, with hyper-parameters frozen. Evaluated customers are those who bought in the target week, including cold-start customers (scored on popularity).

3 Baselines first, then candidate models

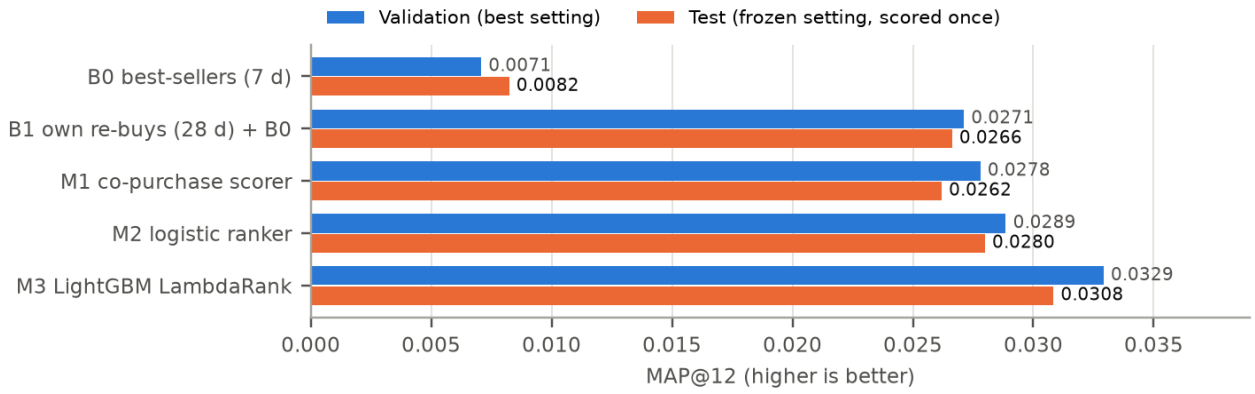


Figure 1. MAP@12 per model. Blue: best validation setting. Orange: test week, setting frozen. The order holds on test except that M1 drops below B1. Every tuned model loses a little from validation to test, the expected optimism of selecting on one week; B0 rises slightly.

Model	Tuning approach	Runs	Selected setting	Valid MAP@12	Test MAP@12	Test Recall @12
B0 best-sellers (7 d)	None (rule fixed in advance)	1	window 7 d	0.0071	0.0082	0.0267
B1 own re-buys (28 d) + B0	None (rule fixed in advance)	1	window 28 d	0.0271	0.0266	0.0563
M1 co-purchase scorer	Full grid 3×3×3: co-purchase window {4, 8, 12} wk × co-purchase weight {0.5, 1, 2} × popularity weight {0.1, 0.3, 1}	27	window 12 wk, CF weight 1.0, pop weight 0.1	0.0278	0.0262	0.0551
M2 logistic ranker	Grid over C {0.01, 0.1, 1}; negatives sampled 20%, seed 42	3	C = 1.0	0.0289	0.0280	0.0597
M3 LightGBM LambdaRank	Random search, 12 trials (sampler seed 42) over leaves, learning rate, trees, min child, column and row sampling; best re-run with seeds 1-3	15	num_leaves 63, learning_rate 0.0337, n_estimators 200, min_child_samples 20, colsample_bytree 0.8, subsample 1.0	0.0329	0.0308	0.0613

4 Is the gain real? Tuning spread, seeds and sampling error

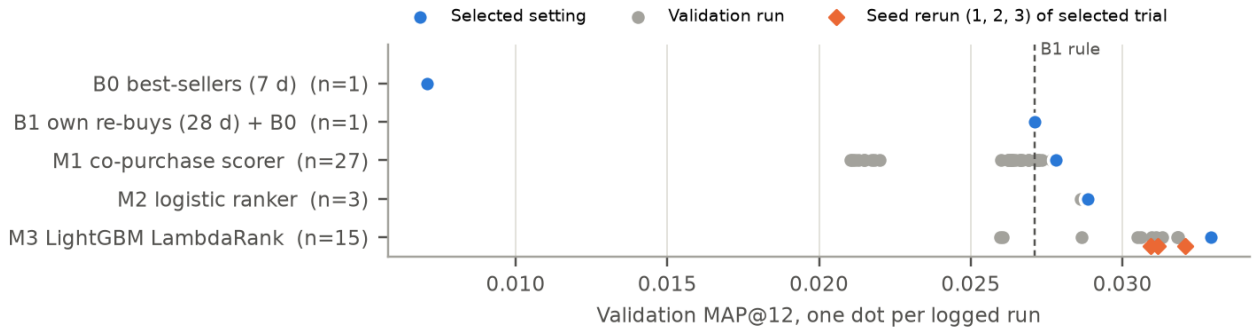


Figure 2. Every validation run. The grey spread is what tuning bought; the dashed line is the untuned B1 rule. Most M1 settings sit at or below B1. M3 beats B1 in 10 of 12 trials and in every seed rerun.

Seed variance. The selected M3 setting scores 0.0309 to 0.0329 across seeds 42, 1, 2, 3 on validation (column subsampling is the only random step). That range (≈0.0020) is wider than the gap between the best and fifth-best trial, so we read the search as finding a plateau, not a sharp optimum. **Sampling error.** A paired bootstrap over test customers (2,000 resamples, seed 7) gives:

Comparison (test)	Difference in MAP@12	95% CI	Reading
M3 LightGBM - B1 rule	+0.0042	+0.0022 to +0.0063	Clear gain
M3 LightGBM - M2 logistic	+0.0028	+0.0010 to +0.0049	Clear gain
M3 LightGBM - M1 co-purchase	+0.0046	+0.0026 to +0.0068	Clear gain
M1 co-purchase - B1 rule	-0.0004	-0.0016 to +0.0007	No detectable difference

5 Who the model serves well (test week)

Group (test)	Customers	B0	B1	M1	M2	M3
Visit days before: 0 (cold)	566	0.0062	0.0062	0.0062	0.0056	0.0075
Visit days before: 1-2	619	0.0066	0.0212	0.0225	0.0239	0.0276
Visit days before: 3-9	1769	0.0059	0.0296	0.0293	0.0299	0.0357
Visit days before: 10+	3866	0.0098	0.0291	0.0283	0.0310	0.0325
Age 18-24	1764	0.0088	0.0254	0.0245	0.0251	0.0271
Age 25-34	2181	0.0094	0.0288	0.0287	0.0301	0.0323
Age 35-44	672	0.0059	0.0260	0.0231	0.0279	0.0312
Age 45-54	1290	0.0066	0.0225	0.0211	0.0241	0.0261
Age 55-64	685	0.0087	0.0293	0.0328	0.0334	0.0391
Age 65+	195	0.0086	0.0259	0.0261	0.0288	0.0390
Age unknown	33	0.0008	0.0703	0.0762	0.0801	0.0896

M3 is best in every activity tier and every age band, but the cold-start tier (566 customers) stays near the popularity floor for all models, because none has history to use (D3 I6). The 65+ and unknown-age bands have few customers (195 and 33); read their scores as indicative only. Top features by gain: rep_days 16%, pop28 12%, user_recency 7%, product_type_no 7%, type_share 6%. Recency of the customer's own purchase leads, consistent with the re-buy finding.

6 Experiment log and reproducibility

experiment_log.csv holds all 52 runs (47 validation, 5 test). Each row has: run id, UTC timestamp, stage, evaluated week, model, data version (data-v1.0 plus the transactions file's SHA-256), feature list, parameters (JSON), random

seed, MAP@12, Recall@12, candidate recall, customers evaluated, runtime, design hash and code hash. Rules and M1 are deterministic (seed 'n/a'). **Independent check: validate_metrics.py** re-derives B0 and B1 with separate vectorised code that does not import the project library; it matches the log on both weeks (valid B1 0.027120, test B1 0.026625; match = True). The bootstrap code was added after tuning and before the test run; the code hash on the test rows records this.

Run it: conda env create -f environment.yml (Python 3.13.5, pandas 2.2.3, scikit-learn 1.6.1, LightGBM 4.7.0, pinned in requirements.txt), place data-v1.0 at ../D2/data-v1.0 (the loader stops if its checksum differs), then **python -I run_experiments.py --stage tune**, **--stage test**, **python -I validate_metrics.py**, **python -I build_report.py**. Tuning took under 3 minutes and the test run under 1 minute on a 16-thread laptop (log timestamps). The data itself stays out of the repository (Kaggle terms, as in D2).

7 Limits and next steps

(1) Candidate recall is 9.6%: 9 in 10 purchased items never reach the ranker. Next: add same-product-type and new-arrival candidates and measure recall before re-tuning. (2) One validation week is a thin basis for selection; a rolling origin over the last four weeks [2] would steady it. (3) The data is an H&M proxy: channel meaning, gender and Westside's catalogue are unknown, so the scores show a method, not a forecast of in-store uplift. (4) The outfit matcher and colour advice (M5) need their own labelled data; purchase logs cannot validate them (D3 I4, I8).

References

[1] Kaggle (2022). H&M Personalized Fashion Recommendations, evaluation page (MAP@12). [2] Hyndman, R. J. & Athanasopoulos, G. (2021). Forecasting: Principles and Practice, 3rd ed., §5.10 Time series cross-validation. OTexts. [3] Kapoor, S. & Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. Patterns, 4(9). [4] Sarwar, B., Karypis, G., Konstan, J. & Riedl, J. (2001). Item-based collaborative filtering recommendation algorithms. WWW '01. [5] Burges, C. J. C. (2010). From RankNet to LambdaRank to LambdaMART: An overview. Microsoft Research TR-2010-82. [6] Ke, G. et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. NeurIPS 30. [7] Ferrari Dacrema, M., Cremonesi, P. & Jannach, D. (2019). Are we really making much progress? RecSys '19.